ACUCOBOL-GT Appendices
Version 9.0.1

Books

Appendix I: Library Routines > I.1 General Syntax and Library List >

I$IO

The I$IO routine provides an interface to the file handler. An operation code and some number of additional parameters (depending on the operation called) are passed to the routine. The return code is set automatically after the call. The external variable "F-ERRNO" is set according to any errors found. "F-ERRNO" may not be reset on entry to I$IO, and should be checked only if I$IO returns an error condition.

Usage

```
CALL "I$IO"
    USING OP-CODE, parameters
```

Parameters

OP-CODE Numeric parameter

Specifies the file handling routine to be performed. This table shows which operation corresponds to each operation code. The operations are detailed in the description below:

Code	Operation
1	OPEN-FUNCTION
2	CLOSE-FUNCTION
3	MAKE-FUNCTION
4	INFO-FUNCTION
5	READ-FUNCTION
6	NEXT-FUNCTION
7	PREVIOUS-FUNCTION
8	START-FUNCTION
9	WRITE-FUNCTION
10	REWRITE-FUNCTION
11	DELETE-FUNCTION
12	UNLOCK-FUNCTION
13	REMOVE-FUNCTION
14	SYNC-FUNCTION
15	EXECUTE-FUNCTION
16	BEGIN-FUNCTION

17	COMMIT-FUNCTION
18	ROLLBACK-FUNCTION
19	RECOVER-FUNCTION
21	IN-TRANSACTION-FUNCTION

parameters Vary depending on the op-code chosen

The remaining parameters vary depending on the operation selected. They provide information and hold results for the operations specified. All parameters are passed *by reference*. Parameters may be omitted from those operations that do not require them. These parameters are detailed in the "Description" below.

Description

All parameters passed to I$IO are passed *by reference*. This applies even to parameters that are integer values in the corresponding file handling routines. All numeric parameters should be passed to I$IO as SIGNED-SHORT values. The I$IO routine provides any necessary addressing conversions. Note that a parameter must be in the correct format for its type. Parameters that are PIC X must be terminated by a LOW-VALUES character.

Except for the MAKE function, I$IO will automatically terminate any PIC X parameters with a LOW-VALUES byte for you. Also, you do not have to specify SYNC for level 01 or level 77 parameters because they are automatically synchronized by ACUCOBOL-GT.

The file "filesys.def" is a COBOL COPY file that contains many useful definitions for use with I$IO. It contains definitions for the I$IO codes along with the "F-ERRNO" error values and many useful pre-declared variables that are of the proper type and usage.

The behavior of this routine is affected by the FILENAME_SPACES configuration variable. The value of FILENAME_SPACES determines whether spaces are allowed in a file name. See the entry for FILENAME_SPACES in Appendix H for more information.

Note: The runtime configuration variable FILE_PREFIX is ignored by the I$IO routine.

OP-CODES and PARAMETERS

OPEN-FUNCTION (op-code 1)

CLOSE-FUNCTION (op-code 2)

MAKE-FUNCTION (op-code 3)

INFO-FUNCTION (op-code 4)

READ-FUNCTION (op-code 5)

NEXT-FUNCTION (op-code 6)

PREVIOUS-FUNCTION (op-code 7)

START-FUNCTION (op-code 8)

WRITE-FUNCTION (op-code 9)

REWRITE-FUNCTION (op-code 10)

DELETE-FUNCTION (op-code 11)

UNLOCK-FUNCTION (op-code 12)

REMOVE-FUNCTION (op-code 13)

SYNC-FUNCTION (op-code 14)

EXECUTE-FUNCTION (op-code 15)

BEGIN-FUNCTION (op-code 16)

OPEN-FUNCTION (op-code 1)

This routine opens an existing indexed file. If it is successful, the value in RETURN-CODE should be moved to a data item that is USAGE POINTER. This data item is passed as the open file handle to the other file handling routines. If it fails, RETURN-CODE is set to a NULL value. After the file is opened, the primary key is set as the current key of reference and is positioned at the beginning of the file.

The OPEN routine has three parameters, *name*, *mode* and *l_parms*.

Name is the name of the file to open. It must be null-terminated.

Mode is one of the following values (defined in "filesys.def"):

Finput	open for input only
Foutput	open for output only
Fio	open for input and output
Fextend	same as Foutput

This routine only opens already existing files. If the file does not exist, the routine fails, even when opening with mode "Foutput".

"Foutput" does *not* delete the current file (this is different from the OPEN OUTPUT verb in COBOL).

Mode may furthermore have one of the following flags added to it to indicate file locking options (defined in "filesys.def"):

Fread_lock	locks file against other updaters
Fwrite_lock	locks file against all others
Fmass_update	same as Fwrite_lock
Ftrans	extended locking rules for transaction management

Some file systems cannot support the "Fread_lock" option correctly. For these systems, the setting of the external variable "F_UPGRADE_RLOCK" determines what happens. If this variable is set to the default, then the "Fread_lock" setting is treated as a normal open (no file locking). If this variable is non-zero, then the "Fread_lock" setting is treated as "Fwrite_lock" instead.

A few file systems do not support any form of file locking. If locking is requested on one of these file systems, the open proceeds as if file locking were not specified, but the external variable "F-ERRNO" is set to W_NO_SUPPORT. This is also returned for file systems that cannot support multiple record locks when "Fmulti_lock" is specified.

If "Fmass_update" is used, then the file system is also requested to emphasize speed of updates over file security.

Additionally, "Fmulti_lock" may be also added to *mode* to request that more than one record lock be maintained in the file by this process. If this option is not specified, then any I/O operation on the file will first release any currently locked record. This results in only one record lock being set in the file at any time. When this option is used, locked records are released only when the file is closed or when the UNLOCK routine is called.

L_parms points to a null-terminated string that describes the key structure for the file. The *l_parms* parameter is the same as the *l_parms* parameter passed when using the MAKE op-code. This parameter is a string that contains three comma-separated numbers. These values are (in order):

1 the maximum record size

2 the minimum record size

3 the number of keys for the file.

If the maximum record size does not match the minimum record size, then variable sized records are implied. This parameter is not used by all file systems, but is supplied for those file systems that cannot determine these values on their own.

Note: The *l_parms* parameter is always required, but it is ignored by the default file systems shipped with ACUCOBOL-GT (RMS for VAX/VMS machines and Vision for all others). It is required by other file systems, however, including Btrieve and C-ISAM. If you ever intend on using any of these file systems, you should ensure that the values passed in *l_params* are correct.

CLOSE-FUNCTION (op-code 2)

This routine closes an open file. It also removes currently held locks on the file. The CLOSE routine has only one parameter, *f*, a file handle returned by OPEN. For some file systems, it is possible that CLOSE will write additional records that had been previously buffered by the system. For this reason, it is possible that a "disk full" condition can occur.

MAKE-FUNCTION (op-code 3)

This routine is used to create a new indexed file. It will overwrite any existing file. This routine will not overwrite a file that is currently in use. If the file is in use, the error E_FILE_LOCKED will be returned. The MAKE routine has six parameters, *name*, *comment*, *p_parms*, *l_parms*, *keys*, and *trans*.

This routine does not automatically terminate its parameters with LOW-VALUES, you must insure that the parameters are correctly terminated yourself. If you do not wish to supply an alternate collating sequence in the "trans" parameter to MAKE, then simply omit the parameter. The I$IO routine does not allow you to specify a "NULL" parameter which is what is expected by the MAKE routine. Instead, by omitting the parameter, the I$IO routine will construct a "NULL" parameter for you.

The host file system may ignore any or all of the values specified by *comment*, *p_parms* and *trans*. If the host system cannot perform the requested operation, it is ignored (however, if the *trans* value is ignored, "F-ERRNO" is set to W_NO_SUPPORT). If the host system cannot support one of the values specified in *l_parms* or *keys*, then MAKE fails and RETURN-CODE is set to the error condition E_NO_SUPPORT. If any of the parameters are ill-formed or contain values outside of the legal range of values, E_PARAM_ERR is returned.

Name points to the name of the file. This must be null-terminated.

Comment may be NULL or may point to comment string that describes the file. This comment is stored in the file, but has no other functional use. The comment may be up to 30 characters long and must be null-terminated. Many host file systems cannot store comments. On these systems, this parameter is ignored.

P_parms points to a string that describes various physical characteristics of the file. This string consists of five numeric fields separated by commas and must be null-terminated. Each of these fields advises the file system of desired treatment for the physical storage of the file. Individual file systems may ignore some or all of these fields. Furthermore, *p_parms* may be NULL to imply "0" for each of the fields.

The "filesys.def" COPY file has a data item containing these fields.

The fields are as follows:

1. Blocking factor. This value is the number of disk sectors to store in a single file block. It may be zero to request the default blocking factor for the file system.
2. Pre-allocate amount. This is the number of file blocks (the size of which depends on the blocking factor above) to initially allocate to the file. The purpose of this is to allocate contiguous disk space for the file. It does not limit the file's total size. This value may be zero to request the default initial allocation amount.
3. Extension factor. This is the number of file blocks to add to the file when it needs to grow. The purpose of

this is to help reduce disk fragmentation. This value may be set to zero to request the default extension amount.

4. Compression factor. This is a value between zero and 100. It represents the amount of file compression desired. A value of zero indicates no compression and a value of 100 indicates maximum compression. Values in between indicate varying degrees of compression. The exact meaning of this depends on the host file system.
5. Encryption flag. If this value is "1", then encryption is desired for the disk file. If it is zero, then no encryption is desired. The results of encryption depend on the host file system. Since no key is supplied, the results of encryption are usually only moderately secure.

l_parms points to a string that describes various logical characteristics of the file. The string consists of three numeric fields separated by commas. The string must be null-terminated.

The "filesys.def" COPY file has a data item containing these fields.

The fields are as follows:

1. Maximum record size. This is the size of the largest record to be placed in the file. For Vision 5 files, this may range from 1 to 67,108,864. For Vision 2, 3, and 4 files, this may range from 1 to 32,767.
2. Minimum record size. This is the size of the smallest record to be placed in the file. This may range from 1 to the maximum record size. If this field is the same as the maximum record size, then fixed-length records are implied.
3. Number of keys. This is the number of keys in the file, including the primary key. This may range from 1 to MAX_KEYS.

keys -- points to a null-terminated string that describes the key structure for the file. *keys* is a string of numbers separated by commas. The first key described is the primary key. It may not allow duplicate values. The primary key is called key "0". The next key described is key "1" and so on. There should be as many keys described as the "number of keys" field of *l_parms* indicates.

The "filesys.def" COPY file has a data item containing these fields.

Each key description contains the following information:

1. Number of segments. This is the number of segments in this key. A segment is a contiguous region of bytes in the record. You may describe a split key by specifying more than one segment. With Vision Version 3 files, this may range from 1 to V3_MAX_SEGS. With Vision Version 4 and 5 files, this may range from 1 to MAX_SEGS.
2. Duplicates flag. If this value is "1", then duplicate keys are allowed. If "0", then duplicate values are not allowed. This must be "0" for the primary key.
3. Segment size. This is the number of bytes in the first segment. This may be between 1 and MAX_KEY_SIZE.
4. Segment offset. This is the offset from the beginning of the record to the first byte of the segment. A segment that starts at the beginning of the record has an offset of "0".
5. Remaining segments. The segment size and segment offset fields are repeated for each additional segment in the key. The sum of the segment sizes may not exceed MAX_KEY_SIZE.

For example, a file with two keys, the first one having two segments (offset zero, length 10 and offset 50, length 5) and the second one with one segment (offset 20, length 15) and allowing duplicates would be written:

```
2,0,10,0,5,50,1,1,15,20
```

trans -- This parameter specifies an alternate collating sequence for the keys. If this parameter is NULL, then keys are ordered in ascending sequence based on their native unsigned value. If *trans* is not NULL, it must point to a 256 byte region of memory. Unlike other strings, this need not be null-terminated and is likely to contain nulls within it. This 256 byte region is used as a translation table on the bytes of each key to arrive at a new key-ordering. Each byte is used as an index into this table, and the resulting value is used to order the keys. Some host file systems cannot do key translations. On these systems, this parameter is ignored.

INFO-FUNCTION (op-code 4)

This routine returns a variety of information about the open indexed file *f*, depending on the value of *mode*. The information is returned in the area pointed to by *result*. Most of the *modes* return one or more numeric values. These values are represented in *result* as fixed size fields with the numbers represented as strings. When more than one value is returned in *result*, the values are separated by commas.

The "filesys.def" COPY file contains layouts for each kind of information that can be retrieved with this routine.

The INFO routine has three parameters, *f*, *mode*, and *result*.

F is a file handle returned by OPEN.

Mode determines what ***result*** is returned with a series of comma-separated numbers that define the format of *result*. (Note that in the following descriptions, the first value is number "1", the second is number "2" and so on. The number of times the field number is repeated represents the size of the field. For example "111,22" indicates that two values are returned, the first one is three digits long and the second one is two digits long.)

When a particular *mode* cannot be determined by the file system, then the error value E_NO_SUPPORT is set. If a particular value of mode "-1" or mode "-2" cannot be determined, it is set to zero.

The following *modes* are supported:

-1 This returns the same information as the *l_parms* parameter of the MAKE function. *Result* is in the format of "11111,22222,333" where:

1 = maximum record size

2 = minimum record size

3 = number of keys

-2 Returns the same information as the *p_parms* parameter of the MAKE function. *Result* is in the format of "11,22222,33,444,5" where:

1 = blocking factor

2 = pre-allocation amount

3 = extension factor

4 = compression factor

5 = encrypted flag

-3 Returns the comment specified to the MAKE function that made the file. The format is a null-terminated string of up to thirty characters.

-4 Returns the number of records in the file. This is returned as a 10-digit number.

-5 Returns the 256-byte key translation table specified to MAKE when the file was originally made. If no key translation table was specified, then the E_NO_SUPPORT error is set. In this case, this should be simply taken to mean that the native key ordering was used.

-6 Returns the number of currently locked records for the file handle passed to I$IO.

-7 For Vision Version 4 and 5 files, returns the number of data and index segments in the form: "11111,22222" where

1 = number of data segments

2 = number of index segments

For Vision Version 2 and 3 files, this mode always returns "00000,00000".

-8 Returns the name and size of a Vision Version 4 or 5 file segment.

This mode is different from the other modes in that it uses the third argument as both INPUT and OUTPUT. Set FS-TYPE to FS-DATA (255) or FS-INDEX (254) and FILE-SEGMENT-NUMBER to the number of the segment you want information about. Upon return, name will contain the filename of the segment and FS-SIZE will contain the size of the segment. The FILE-SEGMENT-INFO data item in filesys.def describes this structure.

This operation can be called with a Vision Version 2 or 3 file, also. In this case, type is ignored, but seg must be 1. The name and size returned will be those of the single Version 2 or 3 file.

-9 Returns the sum of the sizes of all the segments that make up a Vision Version 4 or 5 file. The return value is 15 digits long. If called with a Vision Version 2 or 3 file, this operation returns the size of the single Version 2 or 3 file.

-10 Returns the version number of the Vision file in a three character string. For example, if the file is Vision Version 5, "005" is returned.

0+ A *mode* of zero or greater indicates that information about a particular key is desired. That key information is returned as "11,2,333,44444" where

1 = number of segments in key

2 = "1" if duplicates are allowed

3 = size of first segment

4 = byte offset of first segment

The third and fourth fields are repeated for each additional segment in the key.

Note: Versions of INFO prior to the version used by the 3.2 runtime returned only one digit for the number of segments in the key.

READ-FUNCTION (op-code 5)

This routine reads a record out of an indexed file. The READ routine has three parameters, *f*, *record*, and *keynum*.

F must be a valid file handle returned by OPEN.

Record points to the area to hold the record read.

Keynum is the key number to read from. Key "0" is the primary key, key "1" is the first alternate and so on. The bytes in *record* corresponding to the key *keynum* must contain the key value of the record to be read. If this routine succeeds, RETURN-CODE is set to the size of the record read. RETURN-CODE is set to zero on failure.

Records read by a file open for input only are not locked. Furthermore, most file systems do not block the reading of locked records by a file open for input (Note that this feature depends on the host file system - not all can support it). Records read from a file open for I/O are automatically locked unless the external variable "f-no-lock" is set to a non-zero value first in which case they are treated in the same manner as files open for input.

If the key *keynum* allows duplicates and the next record in the file contains the same key value as the record read, the variable "F-ERRNO" is set to W-DUP-OK. This feature is not supported by many host file systems.

A successful READ causes the current key of reference to be set to *keynum* and the file position is set to the record read. This is used by the NEXT and PREVIOUS routines. If READ is unsuccessful, then the current key of reference is set to an undefined state.

NEXT-FUNCTION (op-code 6)

This routine reads the next record in the sequence of records specified by the current key of reference. When a file is first opened, its key of reference is the primary key and the file is positioned so that the NEXT record is the first record in the file. The READ and START routines can change the current key of reference. The NEXT routine has two parameters, *f*, and *record*. *F* must be a valid file handle returned by OPEN. *Record* points to the area to hold the record read.

If this routine succeeds, RETURN-CODE is set to the size of the record read. If it fails, RETURN-CODE is set to zero and sets the current key of reference to the "undefined" state. However, if it fails due to the record being locked, the current key of reference is unchanged and the file pointer is set to the locked record (some file systems do not support this rule). Also, if it fails because it reached the end of the file, then the current key of reference is left unchanged and the file pointer is positioned so that a call to PREVIOUS returns the last record in the file.

The NEXT routine follows the same record locking rules as the READ routine.

If the record following the one read contains the same key value (in the current key of reference), then "F-ERRNO" is set to W-DUP-OK.

PREVIOUS-FUNCTION (op-code 7)

This routine behaves just like the NEXT routine with two exceptions. The first is that RETURN-CODE is set to the preceding record in the file instead of the next one. The second is that it never sets the W_DUP_OK error value. The PREVIOUS routine has two parameters, *f*, and *record*. *F* must be a valid file handle returned by OPEN. *Record* points to the area to hold the record read.

Some file systems do not support the ability to read a file backwards. For these systems, this routine sets the error value E_NO_SUPPORT.

This routine treats the beginning of the file in a manner that is analogous to the way NEXT treats the end of the file. If an PREVIOUS call reaches the beginning of the file, a call to NEXT returns the first record of the file.

START-FUNCTION (op-code 8)

This routine selects the current key of reference and positions the file pointer for the next NEXT or PREVIOUS routine. The START routine has five parameters, *f*, *record*, *keynum*, *keysize* and *mode*.

F must be a valid file handle returned by OPEN.

Record points to the area to hold the record read.

Keynum selects which key to use. The corresponding key area in *record* must contain the key value that will be used to position the file.

Keysize indicates the size of the key. If *keysize* is zero, the entire key is used. Otherwise, only the first *keysize* bytes of the key will be used.

Mode selects how the file is to be positioned with respect to the key value defined in *record*. It can be one of the following values:

F_EQUALS	The file is positioned at the record that matches the key value

F_NOT_LESS	The file is positioned at the record that matches the key value, or the next greater one if no one matches
F_GREATER	The file is positioned at the first record greater than the key value specified
F_LESS	The file is positioned at the last record smaller than the key value specified
F_NOT_GREATER	The file is positioned at the record that matches the key value, or the last record smaller than the key value if no one matches

The F_EQUALS mode is usually used to test for the existence of a record or to position a file when the key value is known. The F_NOT_LESS and F_GREATER modes are used to position the file for a series of NEXT calls and the F_LESS and F_NOT_GREATER modes are used to prepare for a series of PREVIOUS calls.

After a successful START, the current key of reference will set to *keynum*. The next NEXT or PREVIOUS call will return the record selected by the START routine. Note that in this case, NEXT and PREVIOUS both return the same record.

If the START routine fails, then the current key of reference is placed in the "undefined" state.

Some file systems cannot perform the F_LESS or F_NOT_GREATER modes. On these file systems, specifying these modes causes START to return an error and set the E_NO_SUPPORT condition.

WRITE-FUNCTION (op-code 9)

This routine adds a new record to the passed file. The WRITE routine has three parameters, *f*, *record*, and *size*.

F must be a valid file handle returned by OPEN.

Record points to the record to add.

Size is the size of the record. If *size* is zero, then the maximum record size for the file is used.

If the file has keys that allow duplicate values, and one or more of those keys in *record* match keys that already exist in the file, then "F-ERRNO" is set to W-DUP-OK. Several file systems cannot detect this condition and in this case, "F-ERRNO" is set to zero instead.

For keys that allow duplicates, the new record is added such that its keys are placed at the end of the sequence of those keys with the same value. Many file systems do not support this rule, in which case the ordering is defined by the file system.

The WRITE routine does not change the current file position or affect the current key of reference.

REWRITE-FUNCTION (op-code 10)

This routine replaces an existing record in the file. The REWRITE routine has three parameters, *f*, *record*, and *size*.

F must be a valid file handle returned by OPEN.

Record points to the new record to place in the file.

Size may be zero to indicate the maximum record size for the file. The record replaced is specified by the primary key value found in *record*. The size of the new record need not match the size of the existing record.

For keys that are unchanged between the new record and the old record, the ordering of those keys is unchanged. For keys that have changed, the new keys are placed at the end of the sequence of keys that have the same value. If any of the changed keys match keys that already exist in the file (and these keys allow duplicates), then "F-ERRNO" is set to W-DUP-OK. Note that these rules depend on the host system and may be different for some file systems.

The REWRITE routine does not affect the file position or the current key of reference.

DELETE-FUNCTION (op-code 11)

This routine deletes the record identified by the value found in the primary key area of *record*. It does not affect the current file position or key of reference. The DELETE routine has two parameters, *f*, and *record*. *F* must be a

valid file handle returned by OPEN. *Record* points to the area to hold the record read.

UNLOCK-FUNCTION (op-code 12)

This routine unlocks any locked records held by the current process in the passed file. It is not an error to call this routine when there are no locked records. This routine does not affect the current file position or key of reference. This routine will not unlock any records if it is called during a transaction. "Commit" (see below) should be used instead. The "unlock" routine has only one parameter, *f*. *F* must be a valid file handle returned by OPEN.

REMOVE-FUNCTION (op-code 13)

This routine should remove from disk the indexed file indicated by *name*. This routine is specialized for indexed files because some host systems implement indexed files in more than one physical file. This routine should be called only when the file to be removed is closed. It has only one parameter, *name*. *Name* points to the name of the file. It must be NULL terminated.

SYNC-FUNCTION (op-code 14)

This routine causes all file buffers to be flushed to disk. The SYNC routine has only one parameter, *all_files*, which is a bit field. If

```
(all_files & FA-MASS-UPDATE)
```

is non-zero, then MASS-UPDATE files should be synced. If

```
(all_files & FA-REMOTE)
```

is non-zero, then remote files should be synced.

The constants FA-MASS-UPDATE and FA-REMOTE are defined in "filesys.def".

The exact effect of this routine depends on the host file system. It is entirely possible that this routine will do nothing under a particular system. Because of the highly host-dependent nature of this routine, no error reporting is done.

EXECUTE-FUNCTION (op-code 15)

This routine executes a file-system specific command. The EXECUTE routine has two parameters, *system*, and *command*.

System points to the name of the file system (e.g. "vision", "btrieve", etc.).

Command contains a command to be executed by that file system. The list of available commands is completely file-system specific. Many file systems do not have any commands that can be executed in this manner. The routine returns 0 if successful, otherwise, RETURN-CODE is set to a file-system specific error value. For file systems that do not support any commands, "-1" is always returned.

Note: Different file systems process this routine differently. For example, the interface for Microsoft SQL Server has some unique parameters. See the documentation for the specific file system interface for exceptions to this operation.

BEGIN-FUNCTION (op-code 16)

The BEGIN routine initiates a transaction. On indexed file systems this usually opens the log file for appending the first time it is called (if the log file doesn't exist, it is created). This routine has no parameters.

COMMIT-FUNCTION (op-code 17)

This routine commits all changes and releases all locks. It also ends a transaction. This routine has no parameters.

ROLLBACK-FUNCTION (op-code 18)

This routine rolls back all files affected to the state that they were in after the last completed transaction. This routine has no parameters.

RECOVER-FUNCTION (op-code 19)

This routine rolls forward all files affected to the state that they were in after the last completed transaction. This

routine has no parameters.

IN-TRANSACTION-FUNCTION (op-code 21)

This routine returns a value indicating whether or not the program is currently in an unfinished transaction. The return value is "1" if there is current and unfinished transaction, "0" otherwise. This routine has no parameters.

Example

The following program opens a file, determines the number of records in the file, and then reads each of those records, printing out the size of each record found.

This example makes heavy use of variables found in "filesys.def".

```
IDENTIFICATION DIVISION.
PROGRAM-ID.  EXAMPLE.

DATA DIVISION.

WORKING-STORAGE SECTION.

* Assume that we already know the maximum record size, minimum
* record size and the number of keys in the file.

78 MAX-SIZE    VALUE 1000.
78 MIN-SIZE    VALUE 100.
78 KEY-COUNT   VALUE 1.

COPY "filesys.def".

77  FILE-HANDLE              USAGE POINTER.
01  RECORD-AREA.
    03 OCCURS MAX-SIZE TIMES  PIC X.

PROCEDURE DIVISION.
MAIN-LOGIC.
* Open the file
    SET OPEN-FUNCTION TO TRUE.
    MOVE MAX-SIZE TO MAX-REC-SIZE.
    MOVE MIN-SIZE TO MIN-REC-SIZE.
    MOVE KEY-COUNT TO NUM-KEYS.
    MOVE Finput TO OPEN-MODE.
    CALL "I$IO" USING IO-FUNCTION, "MYFILE", OPEN-MODE,
                LOGICAL-INFO.
    IF RETURN-CODE = ZERO
        DISPLAY "Could not open file, error code = ", F-ERRNO,
                CONVERT
        STOP RUN.
    MOVE RETURN-CODE TO FILE-HANDLE.
* Now get the record count
    SET INFO-FUNCTION TO TRUE.
    SET GET-RECORD-COUNT TO TRUE.
    CALL "I$IO" USING IO-FUNCTION, FILE-HANDLE, INFO-MODE,
                RECORD-COUNT-INFO.
    IF E-NO-SUPPORT
        DISPLAY "File system cannot determine record count"
        PERFORM CLOSE-FILE
        STOP RUN.
* Read each record
    SET NEXT-FUNCTION TO TRUE
    PERFORM NUMBER-OF-RECORDS TIMES
        CALL "I$IO" USING IO-FUNCTION, FILE-HANDLE, RECORD-AREA
        IF RETURN-CODE = ZERO
        DISPLAY "Error reading record, code = ", F-ERRNO,
                 CONVERT
        PERFORM CLOSE-FILE
        STOP RUN
```

```
            ELSE
            DISPLAY "Record size = ", RETURN-CODE, CONVERT, LEFT
            END-IF
        END-PERFORM.
    * All done
        PERFORM CLOSE-FILE.
        STOP RUN.

    CLOSE-FILE.
        SET CLOSE-FUNCTION TO TRUE.
        CALL "I$IO" USING IO-FUNCTION, FILE-HANDLE.
```
